DevOps Engineer Practical Task: Advanced Kubernetes Deployment with GitOps & Helm

Objective

The goal of this task is to demonstrate your ability to create a local Kubernetes cluster, manage core tooling (Argo CD) with Terraform, package applications with Helm, and manage the entire application lifecycle with a GitOps workflow, including stateful workloads and operational tasks.

Technologies to Use

- **Local Kubernetes: k3d** or any other service.
- **Infrastructure as Code: Terraform** (to install and configure Argo CD).
- **GitOps:** Argo CD.
- **Application Packaging:** Helm.
- **Database:** MySQL.
- **Application Images:** Pre-built Backend & Frontend images (**You can use whatever public images you want**, e.g., a simple Go server and an Nginx frontend).
- **Version Control:** Git (provide a link to your public or private Git repository).
- **Containerization:** Docker/Containerd (as the runtime within Kubernetes).

Task Description

1. Local Kubernetes Cluster Setup

- Create a multi-node cluster with **one control-plane node and at least two worker nodes**.
- Verify the cluster is operational (`kubectl get nodes -o wide`) and that your `kubectl` context is correctly configured.

2. Git Repository & Application Packaging (Helm)

- **Structure your Git repository** with the following directories (*you can structure folders however you want, just focus on separation*):
  - `applications/` : This will contain your custom Helm chart for the frontend/backend (alongside frontend and backend application manifests).
  - `infrastructure/` : This will contain the Kubernetes manifests for the backup `CronJob` and for the database.
  - `terraform/` : This will contain all your Terraform code.
- **Create a Custom Helm Chart:**
  - Inside the `applications/` directory, develop a single Helm chart to manage both the **Frontend** and **Backend** applications.
  - The chart must include templates for `Deployments` , `Services` , `ConfigMaps` , etc.
  - The `values.yaml` file should allow for easy configuration of replica counts, image tags, service types, ports, and other configurations.
- **Prepare Infrastructure Manifests:**
  - Database Deployment should be a part of infrastructure
  - Create the manifest for the backup `CronJob` and place it in the `infrastructure/` directory.

3. GitOps Deployment (Terraform & Argo CD)

- **Install Argo CD using Terraform:**
  - In the `terraform/` directory, write Terraform code to install Argo CD into your cluster.
  - Use the Terraform Helm Provider to deploy the official Argo CD Helm chart.
- **Define Argo CD Applications using Terraform:**
  - Using Terraform (e.g., with the `kubernetes_manifest` resource or the community Argo CD provider), define two Argo CD `Application` resources that will automatically sync your repository:
    1. **An `infrastructure` app:** This app must monitor the `infrastructure/` directory in your Git repository. It will be responsible for deploying the MySQL database (using the Bitnami Helm chart and your custom values file) and the backup `CronJob` .
    2. **An `applications` app:** This app must monitor the `applications/` directory in your Git repository. It will be responsible for deploying your custom frontend/backend applications using Helm Chart you have created.
- After running `terraform apply` , both Argo CD and the applications it manages should be deployed and configured.

4. Database Management & Backups

This section remains the same but is now deployed via the `infrastructure` Argo CD app.

- **Deploy & Initialize MySQL:** The `infrastructure` app will deploy MySQL.
- **Automated Backups:** The `infrastructure` app will also deploy the `CronJob` you created. The `CronJob` must:
  1. Run on a schedule (e.g., every 5 minutes for demonstration).
  2. Use a container image with the `mysql-client` .
  3. Execute `mysqldump` to export the database, naming the file with a timestamp.
  4. Securely fetch the MySQL password from the appropriate Kubernetes `Secret` .
  5. Store the backup file onto a **separate `PersistentVolumeClaim`** dedicated to backups.

5. Verification & Documentation

Your `README.md` must provide clear, step-by-step instructions on how to:

1. Set up the local environment.
2. Clone the repository and create the cluster.

3. Creating necessary resources with **Terraform**.
4. Access the Argo CD UI to monitor the `infrastructure` and `applications` apps.
5. Verify that the backup `CronJob` is running and creating backup files on its volume (`kubectl exec` into a pod to check).
6. Access the frontend application and confirm it can communicate with the backend and that data persists in the database.
7. Clean up the entire environment.

Deliverables

1. A link to your Git repository containing:
    - The `terraform/` directory with all code needed to deploy Argo CD and its applications.
    - The `applications/` directory with your applications and helm chart.
    - The `infrastructure/` directory with your MySQL and CronJob manifest.
    - A comprehensive `README.md` file.

Evaluation Criteria

- **Terraform Proficiency:** Clean, effective Terraform code for installing and configuring Helm charts (Argo CD) and Kubernetes resources (Argo CD Applications).
- **Helm Proficiency:** Quality and flexibility of the custom Helm chart.
- **Kubernetes Operations:** Correct implementation of `CronJobs`, `PersistentVolumeClaims`, and `Secrets`.
- **GitOps Workflow:** Correct setup of Argo CD using an "App of Apps" pattern managed by Terraform, monitoring specific repository paths.
- **Database Management:** Successful initialization of the database schema and a working, verifiable backup solution.
- **System Integration:** The entire system functions correctly end-to-end.
- **Documentation:** Clarity and completeness of the `README.md`.